

CHƯƠNG 2 - DOCKERFILE VÀ TỰ TẠO IMAGE

BÀI 4

DOCKER BUILD - TẠO IMAGE my-web:1.0

Bài học thực hành dùng docker build để tạo Docker Image đầu tiên từ Dockerfile và source code, đồng thời quan sát Build Context và Build Cache.

1. Mục tiêu bài học

- Hiểu cú pháp docker build -t my-web:1.0 .
- Tạo được Image my-web:1.0 từ Dockerfile và index.html.
- Hiểu ý nghĩa của Image Name, Tag và dấu . trong lệnh build.
- Quan sát Build Context trong output thực tế.
- Quan sát Build Cache qua trạng thái CACHED.
- Biết docker build tạo Image chứ chưa tạo Container.
- Biết kiểm tra Image bằng docker images.

2. Điều kiện trước khi build

Docker Desktop phải đang chạy để Docker CLI kết nối được Docker Engine. Nếu Docker Desktop chưa chạy, các lệnh như docker build, docker images, docker run hoặc docker ps có thể báo lỗi không kết nối được Docker API.

```
Docker CLI
↓ Docker API
Docker Engine
↓
Build / Run / Images / Containers
```

Lưu ý: Nếu gặp lỗi failed to connect to the docker API hoặc không tìm thấy dockerDesktopLinuxEngine, hãy mở Docker Desktop và chờ Docker Engine chạy.

3. Kiểm tra project trước khi build

Project:

```
E:\Docker\Projects\my-web
├─ Dockerfile
└─ index.html
```

Dockerfile:

```
FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html
```

Trước khi build nên kiểm tra:

```
cd E:\Docker\Projects\my-web
pwd
dir
```

4. Lệnh docker build

```
docker build -t my-web:1.0 .
```

Ta tách lệnh thành bốn phần:

```
docker build
-t
my-web:1.0
.
```

docker build

Yêu cầu Docker xây dựng một Image từ Dockerfile và Build Context.

-t

Tùy chọn dùng để đặt tên và Tag cho Image.

my-web:1.0

my-web là tên Image; 1.0 là Tag.

.

Dấu . nghĩa là thư mục hiện tại. Trong docker build ., Docker dùng thư mục hiện tại làm Build Context.

Ghi nhớ: docker build tạo Image. docker run mới tạo và chạy Container.

5. Build Context trong bài này

Terminal đang đứng tại:

```
E:\Docker\Projects\my-web
```

Khi chạy:

```
docker build -t my-web:1.0 .
```

thì Build Context là:

```
E:\Docker\Projects\my-web
```

Docker có thể sử dụng Dockerfile và index.html trong thư mục này.

6. Quá trình build thực tế

Ảnh dưới đây là lần build thành công của project.

```
PS E:\Docker\Projects\my-web> docker build -t my-web:1.0 .
[+] Building 0.2s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 104B
=> [internal] load metadata for docker.io/library/nginx:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 32B
=> [1/2] FROM docker.io/library/nginx:latest@sha256:abe47724e466aeab9a345d8e46a221c2fa8953c7848bb4a3bd9976a7199f
=> => resolve docker.io/library/nginx:latest@sha256:abe47724e466aeab9a345d8e46a221c2fa8953c7848bb4a3bd9976a7199f
=> CACHED [2/2] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:ca7b0b21f5d1e031a46d0727fff652a5fc1356c6d907900d8e6b762be6605873
=> => exporting config sha256:51babf41177ca1ddfff4b6295e4363ca5aed046567055d3d72d38641299b1eb85
=> => exporting attestation manifest sha256:68c4798e15f56dcc36e504399cf9afcc1e2ab1c98a3babb71b2ecab100a4fda4
=> => exporting manifest list sha256:73f9c95de46c61339bce4723eb635fafa44485196bc84ab1028e9c3e1765aaa5
=> => naming to docker.io/library/my-web:1.0
=> => unpacking to docker.io/library/my-web:1.0
PS E:\Docker\Projects\my-web> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	5e2309035332	25.9kB	9.49kB	U
my-web:1.0	73f9c95de46c	239MB	63.4MB	
nginx:latest	abe47724e466	242MB	66.3MB	U

Hình 1. docker build thành công và docker images hiển thị my-web:1.0.

load build definition from Dockerfile

Docker đọc nội dung Dockerfile để biết các instruction cần thực hiện.

load metadata for docker.io/library/nginx:latest

Docker kiểm tra thông tin của Base Image nginx:latest.

load build context

Docker chuẩn bị Build Context từ thư mục được chỉ định bằng dấu .

[1/2] FROM ... nginx:latest

Docker sử dụng nginx:latest làm Base Image.

[2/2] COPY index.html ...

Docker đưa index.html từ Build Context vào /usr/share/nginx/html/index.html trong Image.

naming to docker.io/library/my-web:1.0

Image kết quả được gán tên my-web và Tag 1.0.

7. Kiểm tra Image bằng docker images

```
docker images
```

Output của bạn đã hiển thị:

```
my-web:1.0
nginx:latest
hello-world:latest
```

Sự xuất hiện của my-web:1.0 xác nhận Image mới đã được tạo thành công.

8. Image my-web:1.0 chứa gì?

```
nginx:latest
+
index.html c a project
↓
my-web:1.0
```

Về logic, Image mới chứa Nginx, filesystem và cấu hình nền của Nginx Image, cùng file index.html của bạn tại thư mục phục vụ web mặc định.

9. Source thay đổi có làm Image tự đổi không?

Không. Image là kết quả của một lần build tại một thời điểm.

```
S a index.html
↓
Image cũ không tự  i
↓
docker build lại
↓
Image mới/cập nhật
```

Nếu sửa source code nhưng không build lại thì container tạo từ Image cũ vẫn dùng nội dung cũ.

10. Build Cache

Khi build lại mà các bước không thay đổi, Docker có thể tái sử dụng kết quả trước đó thay vì thực hiện lại toàn bộ.

```
[+] Building 0.3s (7/7) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 104B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:latest 0.1s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                   0.0s
=> [1/2] FROM docker.io/library/nginx:latest@sha256:abe47724e466aeab9a345d8e46a221c2fa8953c7848bb4a3bd9976a7199f 0.0s
=> => resolve docker.io/library/nginx:latest@sha256:abe47724e466aeab9a345d8e46a221c2fa8953c7848bb4a3bd9976a7199f 0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 32B                                  0.0s
=> CACHED [2/2] COPY index.html /usr/share/nginx/html/index.html 0.0s
=> exporting to image                                          0.1s
=> => exporting layers                                          0.0s
=> => exporting manifest sha256:ca7b0b21f5d1e031a46d0727fff652a5fc1356c6d907900d8e6b762be6605873 0.0s
=> => exporting config sha256:51babf41177ca1ddff4b6295e4363ca5aed046567055d3d72d38641299b1eb85 0.0s
=> => exporting attestation manifest sha256:59ba122501668807c9829a5de0f5768b00fdab15e06bb781a65bc154f0c8d677 0.0s
=> => exporting manifest list sha256:32dd560807a43d2e3c30eacb98c781e0cb6376762d5cf76a6bfe5449fa56a6e3 0.0s
=> => naming to docker.io/library/my-web:1.0                   0.0s
=> => unpacking to docker.io/library/my-web:1.0                0.0s
```

Hình 2. Build lần sau xuất hiện CACHED ở bước COPY.

Dòng:

```
CACHED [2/2] COPY index.html /usr/share/nginx/html/index.html
```

cho biết Docker nhận thấy bước COPY không thay đổi nên sử dụng cache.

Ghi nhớ: Build Cache giúp không thực hiện lại các bước build không thay đổi, từ đó giảm thời gian build.

11. Image Layer - mức cần hiểu ở bài này

Docker Image được cấu tạo từ nhiều layer. Có thể hiểu đơn giản:

```
Base Image layers: nginx
↓
Layer thay đổi do COPY index.html
↓
my-web:1.0
```

Ở giai đoạn này chỉ cần nhớ Image có nhiều lớp và Docker có thể tái sử dụng các lớp/bước không thay đổi trong quá trình build.

12. Build và Run khác nhau

```
Dockerfile + Source
↓
docker build
↓
Image

Image
↓
docker run
↓
Container
```

Sau Bài 4, bạn mới có Image my-web:1.0. Chưa có Container được tạo từ Image này.

13. Lỗi thường gặp và cách kiểm tra

Docker Engine chưa chạy

Dấu hiệu: failed to connect to the docker API. Cách xử lý: mở Docker Desktop, chờ Engine hoạt động rồi kiểm tra docker version.

Không tìm thấy Dockerfile

Kiểm tra pwd và dir. Đảm bảo terminal đang đứng trong thư mục chứa Dockerfile.

Không tìm thấy index.html

Đảm bảo index.html nằm trong Build Context và tên file trong COPY viết đúng.

Build Context sai

Nếu dấu . được dùng ở sai thư mục, Docker có thể không tìm thấy file hoặc phải xử lý nhiều file không cần thiết.

14. Câu hỏi kiểm tra và đáp án

Câu 1. docker build tạo ra Image hay Container?

Tạo Image.

Câu 2. Trong docker build -t my-web:1.0 ., my-web là gì?

Là tên Image.

Câu 3. 1.0 là gì?

Là Tag của Image.

Câu 4. Dấu . là gì?

Là thư mục hiện tại. Trong docker build ., thư mục hiện tại được dùng làm Build Context.

Câu 5. Nếu sửa index.html nhưng không build lại thì Image cũ có tự thay đổi không?

Không. Phải docker build lại để Image phản ánh source code mới.

Câu 6. Build Cache dùng để làm gì?

Tái sử dụng kết quả của các bước build không thay đổi để quá trình build nhanh hơn.

Câu 7. Sau khi docker build xong đã có Container chưa?

Chưa. docker build chỉ tạo Image. Muốn có Container cần dùng docker run.

15. Bài tập thực hành

Bài tập 1 - Build lại không thay đổi source:

```
docker build -t my-web:1.0 .
```

Quan sát xem bước nào xuất hiện CACHED.

Bài tập 2 - Sửa nội dung index.html, ví dụ:

```
Hello Docker Version 2!
```

Sau đó build lại:

```
docker build -t my-web:1.0 .
```

Quan sát xem bước COPY còn CACHED hay không.

Bài tập 3 - Kiểm tra Image:

```
docker images
```

Xác nhận my-web:1.0 tồn tại.

Bài tập 4 - Tự giải thích lệnh sau bằng lời:

```
docker build -t my-web:1.0 .
```

Yêu cầu giải thích đủ: build, -t, tên Image, Tag và Build Context.

16. Tóm tắt cú pháp và câu lệnh

Build Image

```
docker build -t my-web:1.0 .
```

Build Image tên my-web, Tag 1.0, sử dụng thư mục hiện tại làm Build Context.

Liệt kê Image

```
docker images
```

Hiển thị các Docker Image đang có trong local Docker Engine.

Kiểm tra Docker Engine

```
docker version
```

Nếu kết nối bình thường sẽ có cả phần Client và Server.

Kiểm tra thư mục hiện tại

```
pwd
```

Hiển thị đường dẫn hiện tại.

Liệt kê file project

```
dir
```

Dùng để xác nhận Dockerfile và index.html tồn tại.

17. Thuật ngữ cuối bài

Docker Build

Quá trình tạo Docker Image dựa trên Dockerfile và Build Context.

Image Name

Tên Image, ví dụ my-web.

Tag

Nhãn hoặc phiên bản của Image, ví dụ 1.0.

Build Context

Tập hợp file/thư mục Docker Build có thể truy cập; trong docker build . là thư mục hiện tại.

Base Image

Image nền, trong bài này là nginx:latest.

Build Cache

Cơ chế tái sử dụng kết quả các bước build không thay đổi.

CACHED

Dấu hiệu trong output cho biết Docker đang dùng lại kết quả build trước đó.

Image Layer

Một lớp cấu thành Docker Image. Nhiều bước build có thể tạo hoặc tái sử dụng các layer.

Docker Engine

Thành phần thực hiện các yêu cầu như build Image và chạy Container.

Docker CLI

Công cụ dòng lệnh mà người dùng gõ docker build, docker images, docker run...

Artifact

Kết quả được tạo ra từ quá trình build; trong bài này Docker Image là artifact được build từ project.

18. Sơ đồ tổng kết

```
Source Code: index.html
+
Dockerfile
↓
docker build -t my-web:1.0 .
↓
Image: my-web:1.0
↓
docker run (Bài 5)
↓
Container
```

Ghi nhớ: Sau Bài 4, bạn đã tự build được Docker Image đầu tiên. Bài 5 sẽ dùng chính my-web:1.0 để tạo Container và mở website qua trình duyệt.

Bài tiếp theo: Chương 2 - Bài 5: Chạy my-web:1.0 thành Container